

Storm Sequence Generator (MKM-SS-001)

Wolfram Oracle Benchmark of the Code and its v3.0 Document

MKM Research Labs

Version 0.2 — 08-November-2026

David K Kelly

CONFIDENTIAL — PROPRIETARY

Legal Notice

PROPRIETARY AND CONFIDENTIAL

This document, including all algorithms, models, methodology, formulas, calculations, and intellectual concepts contained herein, constitutes the exclusive intellectual property and confidential trade secrets of MKM Research Labs ("MKM").

Any usage, reproduction, distribution, modification, or disclosure of this document or any portion thereof, without the express prior written authorisation from MKM Research Labs is strictly prohibited and constitutes an infringement of intellectual property rights.

The document is provided on an "as is" basis. MKM Research Labs makes no representations or warranties of any kind, express or implied, regarding its accuracy, reliability, or suitability for any particular purpose.

All rights reserved. © 2021–2026 MKM Research Labs.

Governed by the laws of the United Kingdom.

Contents

1 Executive Summary **3**

2 Model Purpose and Scope **4**

2.1 What is benchmarked 4

2.2 What is not 4

2.3 Earlier issue 4

3 Mathematical Framework **4**

4 Benchmark Method **5**

5 Validation and Backtesting **6**

5.1 The model document's claims 6

6 Model Limitations and Known Weaknesses **7**

7 Document History **7**

1 Executive Summary

This document benchmarks the Storm Sequence Generator (MKM-SS-001): the generators in `src/port/src/storm_multi/`, which draw the 10,000 multi-storm sequences the stress pipeline turns into hydrographs and trains its flood classifiers on. It asks two questions. Does the code compute what its model document (`storm_multi_v3.0.pdf`, v3.0, 8 November 2026) says? And does what the document says hold?

The code computes its documented equations. A reference written from the document's text replays 19,600 generated sequences on the code's own random stream. Every sequence type and storm count matches, and every duration, gap, start time, intensity and aggregate agrees to 1.2×10^{-15}. In bulk, the code's sequence types, storm counts and capped sequences follow the exact law the document's tables imply, within sampling error (Table 1). Wolfram, computing each probability independently, agrees with the reference to 5.0×10^{-31}.

The document's claims hold: 17 of 17 (Table 2).

- **Every gap is at least its minimum.** v3.0 reserves each gap's minimum before it caps the storms. No multi-storm sequence runs its storms back to back; under v2.0, 46% did.
- **The stated rates are exact.** The cap binds on 60.5% of duration draws. Gaps are shortened in 81.1% of multi-storm sequences. Every storm is capped in 63.4%. Table 9's counts (5,225 isolated and so on) are the exact expectation.
- **The correlation is as stated:** a later storm is at least as strong as the first with probability 0.85.
- **The override reaches the generator.** `storm_control.json` changes the sequence probability and the duration table whether it is applied before or after the generator is imported.
- **The paths, command line, outputs, modules and classifier figures** are those in the repository and the data store.

What the benchmark does not settle. The document records that its own durations do not fit the window. The cap binds on 60.5% of draws, above the 50% at which it asks for recalibration, and recalibration is outstanding. The classifiers it cites were trained on v2.0's sequences. Each sequence's `sequence_start`, which the stress pipeline reads as the season, is the time it was generated (the default batch here: 1 distinct month). The document records each of these as a limitation; none is a claim this benchmark could pass or fail.

Evidence. 8 checks, outcome: all pass (Table 1).

- Wolfram: 15.0.0 for Mac OS X ARM (64-bit) (May 26, 2026) via live kernel session (wolframclient).
- Code under test, at commit `7bbb63078a59` (with uncommitted changes).
- Numerics: numpy 1.26.4, scipy 1.15.1, mpmath 1.4.1.
- Generated 2026-11-08 by `python -m tools.wolfram_oracle ss001`.

2 Model Purpose and Scope

2.1 What is benchmarked

The generation path the document describes (§5.1–§5.2), loaded as its own modules:

- `generators/batch_generator.py`: `generate_event_set`, the category and base-intensity draws;
- `generators/sequence_generator.py`: type, count, the cap with its reserved gap minima, gap shortening, placement, intensities, precipitation and the aggregates;
- the duration, gap and intensity samplers, and the tables in `config.port`;
- `utils/validation.py`.

Also tested: the repository the document cites, namely its paths, command line, outputs and override control, and the classifier evidence.

2.2 What is not

- The package's `models/` subpackage (sequence response, spatial correlation, hydrology), which the document places outside the model.
- Whether the sequence types, durations and gaps are realistic. The document cites no observed sequences, and none are tested.
- The classifier itself (MKM-FC-001). Only the figures this document cites from its training summary are checked.

This is a benchmark of the implementation (validation question Q8), not independent validation.

2.3 Earlier issue

Benchmark v0.1 (7 November 2026) tested v2.0 and found that 11 of its 17 claims did not hold. It is kept as evidence of the check. v3.0 is the model and document that answer it.

3 Mathematical Framework

The reference implements the document's equations from its text, in two halves.

Exact. In rational arithmetic:

- the type mix of Eq. 4 and Table 2, weighted by Table 6;
- the storm counts of Table 1;
- the chance that Eq. 6's cap, $(W_P - (n-1)\, g_{\min})/n$, binds;
- the chance that every storm is capped, which holds every gap at its minimum;
- the chance that Eq. 7 triggers Eq. 8's shortening;
- the chance that a later storm is at least as strong as the first (Eqs. 12–13).

Durations and gaps are rounded to whole hours, as the document says.

Replay. Eqs. 4–20 re-run on a `RandomState` seeded as the code's, drawing in the code's order. Timings are exact rationals, so "every storm capped" is decided without rounding; intensities are worked to 50 digits. The first storm's redraw (§3.7) is mirrored so the two streams stay in step.

4 Benchmark Method

The cases.

- The default batch: 10,000 sequences, seed 42, Table 6's weights, the Thames.
- Every category with every forced type: 400 sequences each, 9,600 in all.
- 100 seeds of 10,000 for the type mix (§7.2).

The checks.

- The replay compares each sequence number for number.
- The Monte Carlo compares the code's shares with the exact ones, in standard errors.
- The validator runs on every sequence generated, and on a sequence broken five ways.

The statements.

- Figures (Table 9, §6.2, §6.3, §7.2, Table 8) are judged against the exact law, to the precision the document states them.
- Every gap is judged against its type's minimum, over all 19,600 sequences.
- Q4's override is judged by applying a `storm_control.json` that switches sequences off and shortens every storm. This runs in a separate interpreter, once with the generator imported before the file is applied and once after.
- Paths, command-line flags and files are judged by searching the repository.
- The classifier figures are judged against the training summary in the data store.

Wolfram. Wolfram works from its own `TriangularDistribution` and `UniformDistribution`, independently of the reference:

- the type mix;
- the cap's chance of binding in each configuration, as a single tail of the triangular law (the reference sums the rounded law);
- the chance that every storm is capped, and of shortening, by convolving the rounded laws;
- the correlation, by `Probability` over Eq. 13's mixture.

5 Validation and Backtesting

Table 1: Benchmark checks

Check	Result	Limit	Outcome
Replay, 19,600 sequences: types or storm counts that differ from the reference's	0	0	pass
Replay: worst relative error over durations, gaps, start times, intensities, precipitation, peak positions and Eqs. 16–20	1.2×10^{-15}	1.0×10^{-12}	pass
Type mix over 100 seeds of 10,000 against Eq. 4 and Table 2's exact law (standard errors)	1.07	4	pass
Storm counts in the default batch against the exact law (standard errors)	1.48	4	pass
Multi-storm sequences with every storm capped, against the exact share (standard errors)	1.05	4	pass
§5.4, the validator on all 19,600 sequences generated: invalid	0	0	pass
§5.4, the validator: injected faults caught (late end, overlap, gap count, type, short gap)	5 of 5	5 of 5	pass
Wolfram agrees with the reference	5.0×10^{-31}	1.0×10^{-25}	pass

5.1 The model document's claims

Table 2: The model document's claims, tested

Claim	Stated	Benchmark finds	Outcome
Eq. 4, Tables 2–6, 8	the parameters in `config.port`	all as stated	met
Eq. 5, §3.5	durations and raw gaps are whole hours; a capped duration takes the cap	21,264 of 21,264 uncapped durations are whole	met
§3.1, §3.4.1, §3.5.1	every sequence ends by 156 h, leaving 12 h to drain	latest end 156.0 h; least drainage 12.0 h	met
§3.5, Table 4, §6.3	every gap is at least 24 h (doublet, cluster) or 6 h (persistent); every storm is capped, holding each gap at that minimum, in 63.4% of multi-storm sequences	least gap doublet 24 h, cluster 24 h, persistent 6 h; 63.4% exactly, 62.7% in the default batch	met
Table 8	a later storm is at least as strong as the first with probability 0.85	0.85 exactly; the code 0.849 over 10,810 storms	met
Table 7	P_{base}: Thames 35, Mekong 90, Ha Long 100 mm; 35 mm where a catchment has no storm module	the code: 35, 90, 100; 35 for the Rhine	met
§6.1, Table 9	expected counts 5,225, 1,469, 2,116, 1,190 (isolated, doublet, cluster, persistent)	exactly 5,225, 1,468.75, 2,116.25, 1,190	met
§7.2	pooled over 100 seeds, every type within 1% of Table 9	worst 0.29% off (one seed: 9.3%)	met
§6.2	the cap binds on every catastrophic duration at $n = 5$, and on 60.5% of all draws	100.0%, and 60.45%	met

Table 2: The model document's claims, tested

Claim	Stated	Benchmark finds	Outcome
§6.3	gaps are shortened in 81.1% of multi-storm sequences; 99.0% of moderate-to-severe persistent ones, 100.0% of catastrophic	81.13%; 98.96%, 99.99999998%	met
§5.3	a fixed seed reproduces the batch	it does	met
§5.4, §7.1	the validator finds no invalid sequence	0 of 19,600; it catches 5 of 5 injected faults	met
Q2	10,000 sequences in under 60 s	0.4 s	met
§5.1	the package holds the modules and functions listed	all present	met
§2.2, §5.5	the consumers' paths, the command line and its output files	all exist	met
Q4	`storm_control.json` overrides every table, applied on importing `config` and when the screen saves, whichever was imported first	applied by `config/__init__.py`, `src/routes/trading/control.py`. Applied before the generator is imported, both take effect; after, both take effect	met
§7.3, §7.4	the training summary on file: 11 Thames gauges, mean AUC-ROC 0.9935; GAUGE-1aaf8b08	11 gauges, mean 0.9935; GAUGE-1aaf8b08 present	met

6 Model Limitations and Known Weaknesses

1. **Benchmark, not validation.** The code is shown to compute its documented equations; whether sequences like these occur over the Thames is not tested.

2. **Searches are of this repository,** at the commit above. The classifier evidence is the data store's training summary as found, which predates v3.0.

7 Document History

Date	Version	Description	Author
07-Nov-2026	0.1	First issue: tests the filed v2.0; 11 of its 17 claims fail	David K Kelly
08-Nov-2026	0.2	Tests v3.0; 17 of its 17 claims hold	David K Kelly